

Universidade do Algarve
Faculdade de Ciências e Tecnologia

Relatório Técnico

Projeto Podcastia

Laboratório de Engenharia de Software
Engenharia Informática
Ano Letivo 2025/2026

Docentes

José Barateiro
Marco Giunti

Grupo de Trabalho

PL1-7

Afonso Bitoque	79925
Duarte Cunha	79146
José Tico	76974
Maria Neto	79852

Maio de 2026

Contents

1	Introdução e Visão Geral do Sistema	2
2	Decisões Técnicas e Justificação da Stack	2
2.1	Backend: Spring Boot vs. Node.js (Express)	2
2.2	Frontend: React 19 + Vite vs. Next.js	3
2.3	Base de Dados: H2 Database vs. PostgreSQL	3
2.4	Integração e Desenho de APIs: REST API (Spring MVC) vs. GraphQL	3
2.5	Comunicação em Tempo Real: WebSockets (STOMP) vs. SSE e Long Polling	3
2.6	Geração de Podcasts com IA: Gemini API vs. OpenAI API	4
2.7	Autenticação e Segurança: JWT vs. Sessões com Cookies	4
2.8	Resumo das Tecnologias Utilizadas	4
2.8.1	Backend	5
2.8.2	Frontend	5
3	Arquitetura de Alto Nível	5
3.1	Diagrama de Arquitetura	5
3.2	Componentização Interna do Backend	6
3.3	Modelo C4 da Arquitetura	6
3.3.1	Contexto do Sistema: C4 Nível 1	6
3.3.2	Containers: C4 Nível 2	7
3.3.3	Componentes do Backend: C4 Nível 3	7
3.3.4	Pipeline de Geração de Podcasts	7
4	Modelo de Dados	7
4.1	Diagrama Entidade-Associação (ERD)	8
4.2	Descrição das Entidades Principais	8
5	Instruções de Deployment e Execução	9
5.1	Requisitos Mínimos	9
5.2	Configuração do Servidor Backend	9
5.3	Configuração da Interface Frontend	10
6	Segurança	10
6.1	Modelo de Segurança	10
6.1.1	Autenticação JWT	10
6.1.2	Proteção de Passwords	10
6.1.3	Controlo de Acesso	10
6.2	Vulnerabilidades Conhecidas e Mitigações	11
7	Garantia de Qualidade e CI/CD	11
7.1	Estratégia de Testes	11
7.1.1	Backend: JUnit 5 e Mockito	11
7.1.2	Frontend: Vitest e Playwright	11
7.2	Análise Estática de Código	12
7.3	Integração Contínua com GitHub Actions	12
8	Gestão do Projeto	12
8.1	Metodologia	12
8.2	Progresso por Sprint	13
8.3	Destaques por Sprint	13
8.3.1	Sprints 1 a 4: Fundações	13
8.3.2	Sprint 5: Motor Central	13
8.3.3	Sprints 7 e 8: Sistema Social	13
8.3.4	Sprints 9 e 10: Comunicação e Administração	13
8.3.5	Sprints 11 e 12: Qualidade e Polimento	13

1 Introdução e Visão Geral do Sistema

O Podcastia é uma plataforma para ouvir, organizar e gerar podcasts de forma automática. O sistema permite aos utilizadores ouvir os seus podcasts favoritos e criar novos conteúdos de áudio a partir de notícias obtidas através de feeds RSS, com apoio de Inteligência Artificial.

As principais funcionalidades implementadas são:

1. **Consumo automático de feeds RSS:** o sistema recolhe notícias de fontes externas, como o *Observador* e o *TechCrunch*, em intervalos regulares. Durante este processo, valida os dados recolhidos, como a data de publicação e o autor, e evita a inserção de conteúdos duplicados na base de dados.
2. **Geração assistida de podcasts:** através da API Gemini, o utilizador pode seleccionar artigos recolhidos dos feeds RSS e gerar um podcast com base nesses conteúdos. O sistema processa o texto, cria o guião e disponibiliza o ficheiro final em formato MP3.
3. **Playlist diária automática:** o sistema acompanha os interesses de cada utilizador através de pontos associados às categorias *Desporto*, *Política*, *Finanças* e *Geral*. Todos os dias, às 00:00, é criada uma playlist personalizada com até 20 podcasts ajustados ao perfil do utilizador.
4. **Sistema de chat e mensagens:** a plataforma permite a comunicação direta entre utilizadores, incluindo o envio de mensagens em tempo real e reações com emojis.
5. **Gestão de relações de amizade:** os utilizadores podem seguir e interagir com outros utilizadores da plataforma.
6. **Painel de administração:** os administradores e moderadores podem realizar ações como suspender ou banir utilizadores, consultar registos de auditoria através de `AdminActionLog`, destacar podcasts ou ocultar conteúdos.

2 Decisões Técnicas e Justificação da Stack

A escolha da stack tecnológica foi orientada pela necessidade de robustez no backend (processamento de tarefas agendadas, parsing de feeds e segurança) e dinamismo no frontend (reprodução de áudio, chat em tempo real e reatividade).

PODCASTIA	
Componente	Tecnologia Adotada
Frontend	React 19 + Vite
Backend	Spring Boot 3.4.3 + Java 17
Base de Dados	H2 Database (Local File)
ORM	Hibernate / Spring Data JPA
Documentação API	SpringDoc OpenAPI (Swagger)

2.1 Backend: Spring Boot vs. Node.js (Express)

Decisão: adotou-se o Spring Boot com Java 17.

Justificação: o Spring Boot foi escolhido por oferecer uma estrutura robusta, segura e adequada ao desenvolvimento do backend do Podcastia. A utilização de Java 17 permite detetar vários erros em tempo de compilação, reduzindo problemas comuns em linguagens menos tipadas, como JavaScript.

Além disso, o Spring Boot disponibiliza um ecossistema maduro para injeção de dependências, segurança com Spring Security e persistência de dados com Spring Data JPA. Isto reduz a necessidade de configurar manualmente vários pacotes externos, como aconteceria numa solução baseada em Node.js com Express.

Outro fator importante foi o suporte a tarefas agendadas. A anotação `@Scheduled` permite implementar facilmente processos automáticos, como o consumo de feeds RSS de 2 em 2 horas e a geração diária de playlists às 00:00.

Alternativa comparada: o Node.js com Express poderia oferecer um arranque mais rápido e menor consumo de memória em repouso. No entanto, exigiria mais configuração manual para autenticação,

controle de acessos, validação de dados e gestão de transações. Por isso, para este projeto, o Spring Boot foi considerado uma opção mais adequada.

2.2 Frontend: React 19 + Vite vs. Next.js

Decisão: adotou-se React 19 com Vite.

Justificação: o React foi escolhido por permitir construir uma interface dinâmica e adequada a uma aplicação interativa como o Podcastia. Como a plataforma inclui reprodução de áudio, navegação entre páginas e chat, era importante garantir uma experiência fluida para o utilizador.

O Vite foi usado por oferecer um ambiente de desenvolvimento rápido. Através do *Hot Module Replacement* (HMR), as alterações feitas no código aparecem quase de imediato durante o desenvolvimento, sem necessidade de recompilações demoradas.

A aplicação segue uma arquitetura *Single Page Application* (SPA). Isto permite ao utilizador navegar entre páginas sem recarregar totalmente o site. Esta característica é importante no Podcastia, por exemplo, para manter o leitor de áudio ativo enquanto o utilizador continua a usar outras áreas da plataforma.

Alternativa comparada: o Next.js poderia ser usado caso o projeto precisasse de *Server-Side Rendering* (SSR) ou de uma forte otimização para motores de busca. No entanto, como o Podcastia é uma aplicação privada, usada após autenticação, esses benefícios não eram essenciais. Além disso, o Next.js iria adicionar mais complexidade ao desenvolvimento e ao deployment.

2.3 Base de Dados: H2 Database vs. PostgreSQL

Decisão: adotou-se o H2 Database com persistência num ficheiro local.

Justificação: o H2 foi escolhido por ser simples de configurar e fácil de executar em ambiente local. A base de dados fica integrada na própria aplicação e guarda os dados no caminho `./data/podcastia`.

Esta opção facilita o desenvolvimento e a avaliação do projeto, porque qualquer elemento da equipa ou docente consegue executar o sistema sem instalar um servidor de base de dados externo.

Alternativa comparada: o PostgreSQL seria uma opção mais adequada para um ambiente de produção, por oferecer maior robustez, melhor suporte a acessos concorrentes e maior capacidade de escalabilidade. No entanto, para o contexto académico deste projeto, o H2 é suficiente e torna a execução mais simples.

2.4 Integração e Desenho de APIs: REST API (Spring MVC) vs. GraphQL

Decisão: adotou-se uma REST API com Spring MVC e SpringDoc OpenAPI.

Justificação: o modelo REST foi escolhido por ser simples, conhecido e adequado à estrutura do projeto. Os recursos principais da aplicação, como podcasts, utilizadores e playlists, podem ser organizados de forma clara através de endpoints como `/api/podcasts`, `/api/users` e `/api/playlists`.

Esta abordagem facilita a implementação de operações comuns, como criar, consultar, atualizar e apagar dados. Além disso, integra-se bem com o frontend em React, que comunica com o backend através de pedidos HTTP em formato JSON.

A utilização do SpringDoc OpenAPI permite gerar automaticamente a documentação da API através do Swagger UI. Isto facilita o trabalho da equipa de frontend, porque os endpoints podem ser testados diretamente durante o desenvolvimento.

Alternativa comparada: o GraphQL permitiria ao cliente pedir apenas os campos necessários em cada pedido, reduzindo problemas como *over-fetching*. No entanto, também aumentaria a complexidade do backend, exigindo a definição de esquemas, resolvers e maior cuidado com paginação e desempenho. Para a dimensão do Podcastia, uma REST API é mais simples e suficiente.

2.5 Comunicação em Tempo Real: WebSockets (STOMP) vs. SSE e Long Polling

Decisão: adotou-se WebSockets com STOMP, através do `spring-boot-starter-websocket`.

Justificação: os WebSockets foram escolhidos porque o chat precisa de comunicação em tempo real entre utilizadores. Esta tecnologia permite manter uma ligação aberta entre o cliente e o servidor, permitindo enviar e receber mensagens sem necessidade de criar novos pedidos HTTP a cada interação.

O uso de STOMP sobre WebSockets permite organizar melhor a comunicação. Com este protocolo, as mensagens podem ser publicadas e recebidas através de tópicos, como `/topic/messages`, tornando a estrutura do chat mais clara e fácil de manter.

Alternativas comparadas: o *Server-Sent Events* (SSE) permite comunicação em tempo real, mas apenas do servidor para o cliente. Para um chat, isto não é suficiente, porque o cliente também precisa de enviar mensagens. O *Long Polling* também poderia ser usado, mas é menos eficiente, pois obriga o cliente a fazer pedidos repetidos ao servidor. Por isso, os WebSockets foram a opção mais adequada para esta funcionalidade.

2.6 Geração de Podcasts com IA: Gemini API vs. OpenAI API

Decisão: adotou-se a API Gemini da Google.

Justificação: a API Gemini foi escolhida para apoiar a geração automática de podcasts a partir de artigos recolhidos dos feeds RSS. Esta funcionalidade exige o processamento de blocos de texto relativamente longos, uma vez que o sistema pode ter de resumir ou adaptar vários conteúdos antes de gerar o guião do podcast.

Outro fator considerado foi a disponibilidade da API para testes durante o desenvolvimento. Para um projeto académico, era importante usar uma solução acessível e simples de integrar, sem criar custos elevados para a equipa.

Alternativa comparada: a OpenAI API também poderia ser usada para gerar guiões com boa qualidade textual. No entanto, tendo em conta o contexto académico do projeto, a API Gemini foi considerada mais adequada pela facilidade de utilização, disponibilidade para testes e integração com o fluxo de geração de podcasts.

2.7 Autenticação e Segurança: JWT vs. Sessões com Cookies

Decisão: adotou-se autenticação baseada em JWT.

Justificação: o JWT foi escolhido porque o frontend e o backend funcionam de forma separada. O frontend corre na porta 5173 e o backend na porta 8080. Com JWT, o utilizador recebe um token após iniciar sessão e esse token é enviado nos pedidos seguintes através do cabeçalho `Authorization`.

Esta abordagem permite que o backend valide cada pedido sem guardar sessões em memória ou na base de dados. Assim, o servidor mantém uma arquitetura *stateless*, ou seja, não precisa de guardar o estado da sessão de cada utilizador.

Alternativa comparada: a autenticação com sessões e cookies também poderia ser usada. No entanto, exigiria que o servidor guardasse a sessão do utilizador, por exemplo em memória, numa base de dados ou num sistema como Redis. Além disso, como o frontend e o backend estão em portas diferentes, a gestão segura de cookies e CORS seria mais complexa. Por isso, o JWT foi considerado mais simples e adequado para este projeto.

2.8 Resumo das Tecnologias Utilizadas

Nesta subsecção são apresentadas as principais tecnologias utilizadas no backend e no frontend do Podcastia, incluindo a respetiva versão e função no sistema.

2.8.1 Backend

Tecnologia	Versão	Utilização
Java	17	Linguagem principal do backend.
Spring Boot	3.4.3	Framework principal para desenvolvimento web, segurança, persistência de dados e WebSockets.
Spring Security	6.x	Autenticação com JWT e autorização baseada em papéis.
Spring Data JPA	3.x	Persistência de dados, queries JPQL e suporte a Specifications.
Hibernate	6.x	ORM e geração automática da estrutura da base de dados.
H2 Database	n/a	Base de dados utilizada em desenvolvimento e testes.
JWT	0.11.5	Geração e validação de tokens JWT com HS512.
ROME Library	2.1.0	Leitura e parsing de feeds RSS/Atom.
SpringDoc OpenAPI	2.8.5	Geração automática da documentação da API através do Swagger UI.
Python edge-tts	7.2.8	Síntese de texto em áudio MP3 com voz em português de Portugal.
Google Gemini API	n/a	Geração de guiões narrativos com recurso a LLM.
Maven	3.x	Gestão de dependências e automação do processo de build.

Table 1: Tecnologias utilizadas no backend.

2.8.2 Frontend

Tecnologia	Versão	Utilização
React	19.2	Framework de UI para componentes, hooks e routing.
Vite	7.3	Bundler e servidor de desenvolvimento com HMR.
React Router	7.13	Routing client-side numa arquitetura SPA.
react-hot-toast	2.4	Notificações visuais não intrusivas.
WebSocket STOMP	n/a	Chat privado em tempo real.
Service Worker	n/a	Suporte offline e funcionalidades associadas a PWA.
Vitest	4.x	Testes unitários de componentes.
Playwright	1.45	Testes end-to-end no browser.
ESLint	9.x	Análise estática de código JavaScript.
Prettier	3.3	Formatação automática de código.

Table 2: Tecnologias utilizadas no frontend.

3 Arquitetura de Alto Nível

O Podcastia segue uma arquitetura cliente-servidor. A comunicação principal é feita por REST e, para funcionalidades em tempo real (chat), é usada comunicação via WebSocket.

3.1 Diagrama de Arquitetura

O diagrama seguinte apresenta a organização geral do sistema e a forma como os principais componentes comunicam entre si.

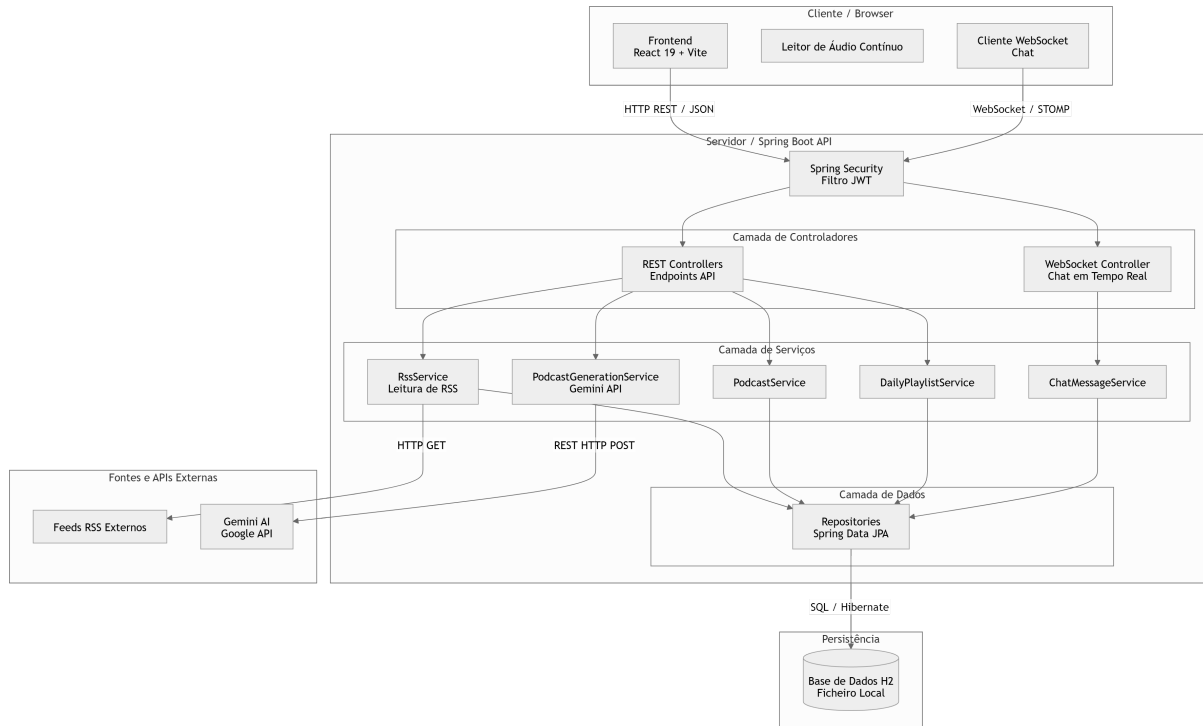


Figure 1: Arquitetura de alto nível do sistema Podcastia.

3.2 Componentização Interna do Backend

O backend está organizado por responsabilidade, separando a API, a lógica de negócio e a persistência.

- **model:** entidades JPA.
- **repository:** acesso a dados via Spring Data JPA.
- **service:** casos de uso e regras de negócio; integrações externas.
- **controller:** endpoints REST e handlers WebSocket.
- **dto:** objetos de transferência entre camadas.
- **config:** configuração transversal (CORS, JWT, WebSockets e OpenAPI).

3.3 Modelo C4 da Arquitetura

A arquitetura do Podcastia é documentada com o modelo C4 (*Context, Containers, Components e Code*). Estes diagramas complementam a descrição da stack na secção anterior e focam-se nas relações entre os principais elementos do sistema. No projeto, os diagramas foram desenhados em Mermaid.js.

3.3.1 Contexto do Sistema: C4 Nível 1

No nível de contexto, o Podcastia interage com quatro sistemas externos principais:

- **Google Gemini API:** usada para gerar o guião narrativo dos podcasts em português.
- **edge-tts em Python:** usado para converter o texto do guião em áudio MP3, utilizando a voz pt-PT-RaquelNeural.
- **Fontes RSS externas:** feeds de parceiros, como *Observador*, *Público Desporto*, *TechCrunch* e *BBC News*, consumidos a cada duas horas.
- **Serviço de email:** usado para envio de notificações e relatórios. Atualmente, encontra-se em modo *stub* para desenvolvimento.

3.3.2 Containers: C4 Nível 2

No nível de containers, o sistema é composto por quatro componentes principais: o frontend, o backend, a base de dados e o sistema de ficheiros.

Container	Tecnologia	Responsabilidade
Frontend SPA	React 19 + Vite	Interface web da aplicação. Comunica com o backend através de REST e WebSocket STOMP.
Backend API	Spring Boot 3.4 + Java 17	Implementa a lógica de negócio, autenticação JWT, agendadores e integrações externas.
Base de Dados	H2 / SQL	Garante a persistência de utilizadores, podcasts, playlists, mensagens e logs.
Sistema de Ficheiros	Disco local	Armazena ficheiros MP3 gerados e imagens de perfil.

Table 3: Containers do sistema Podcastia.

3.3.3 Componentes do Backend: C4 Nível 3

No nível de componentes, o backend segue uma organização em camadas, com dependências do exterior para o interior:

- **Controllers:** expõem a API REST e os endpoints WebSocket, tratam validação básica e delegam para serviços.
- **Services:** implementam casos de uso e orquestram integrações (feeds RSS, geração de podcasts, recomendações, chat e playlists).
- **Repositories:** isolam o acesso à base de dados via Spring Data JPA e as queries necessárias.
- **Config/Security:** configuração transversal (CORS, segurança JWT, WebSockets e OpenAPI) e tarefas agendadas.

Esta divisão reduz acoplamento e facilita manutenção e testes.

3.3.4 Pipeline de Geração de Podcasts

O fluxo de geração de um podcast, desde o pedido do utilizador até à entrega do áudio, é o seguinte:

1. O utilizador envia um pedido POST `/api/podcasts/generate` com o tema e as categorias desejadas.
2. O `PodcastGenerationService` constrói um prompt contextualizado e chama a Google Gemini API.
3. A API devolve um guião narrativo em português europeu.
4. O guião é limpo, removendo a formatação Markdown, e é passado ao processo Python `edge-tts` com a voz `pt-PT-RaquelNeural`.
5. O ficheiro MP3 resultante é guardado em `generated-podcasts/`.
6. Os metadados do podcast são persistidos na base de dados.
7. A resposta inclui o `podcastId` e a URL de streaming imediatamente utilizável.

Nota sobre desempenho O processo de geração é síncrono e pode demorar entre 30 e 60 segundos, dependendo da dimensão do guião gerado e da velocidade da rede. Para podcasts com 3 a 5 minutos de duração, o guião gerado contém tipicamente entre 450 e 750 palavras.

4 Modelo de Dados

O modelo de dados do Podcastia suporta a gestão de utilizadores, a importação de artigos RSS, a geração de podcasts, as playlists diárias, o chat e o registo de ações de administração.

4.1 Diagrama Entidade-Associação (ERD)

O diagrama seguinte apresenta as principais entidades do sistema e as relações entre elas.

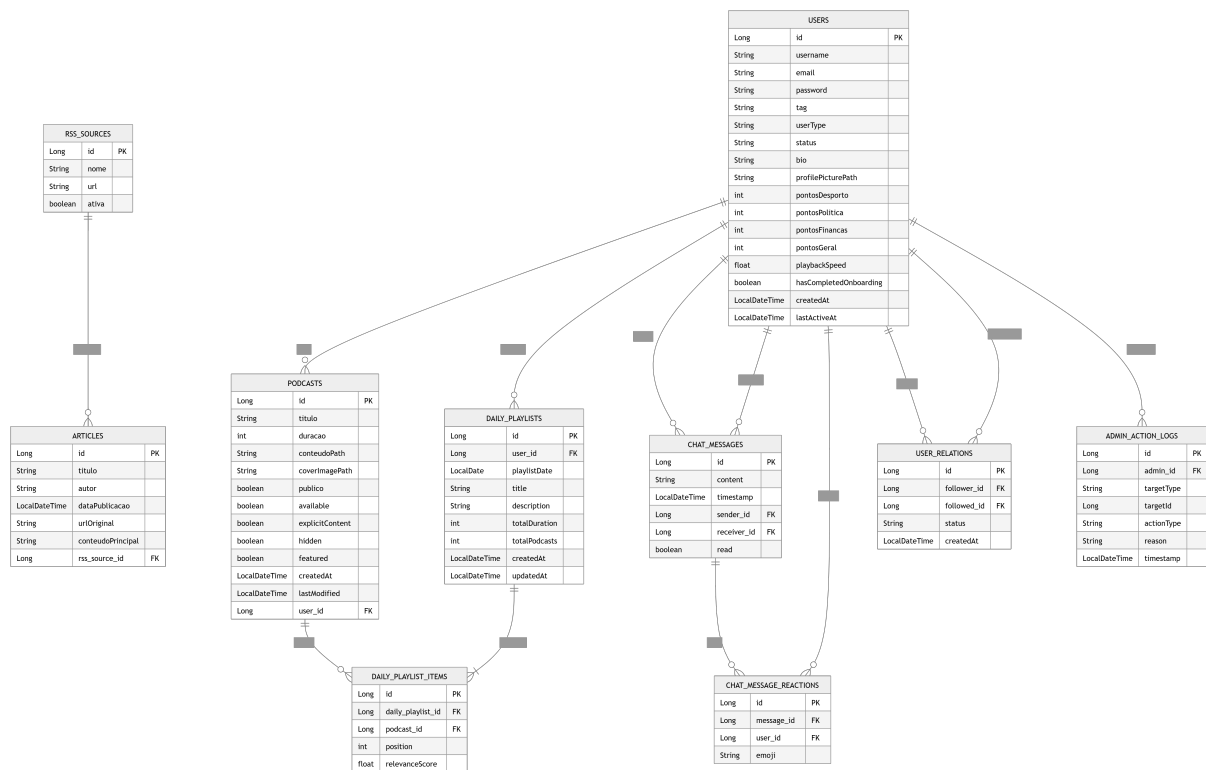


Figure 2: Diagrama entidade-associação do sistema Podcastia.

4.2 Descrição das Entidades Principais

As principais entidades do modelo de dados são:

- **User**: guarda os dados dos utilizadores, incluindo credenciais, informação de perfil, pontuações de interesse por categoria e tópicos escolhidos no onboarding.
- **Podcast**: representa um podcast disponível na plataforma. Guarda informação como o caminho do ficheiro de áudio, duração, imagem de capa, visibilidade e estado de moderação.
- **DailyPlaylist / DailyPlaylistItem**: representam a playlist diária de cada utilizador e os podcasts que fazem parte dela. Também guardam a posição dos podcasts e a pontuação de relevância usada na ordenação.
- **RssSource**: guarda as fontes RSS registadas no sistema, como os endereços dos feeds de notícias.
- **Article**: guarda os artigos recolhidos a partir dos feeds RSS. Estes artigos servem de base para a geração automática de podcasts.
- **ChatMessage / ChatMessageReaction**: representam as mensagens trocadas entre utilizadores e as reações associadas a essas mensagens.
- **UserRelation**: representa as relações entre utilizadores, como seguir outro utilizador ou estabelecer uma ligação dentro da plataforma.
- **AdminActionLog**: guarda o histórico de ações realizadas por administradores, como suspensões, banimentos ou ocultação de conteúdos, incluindo a respetiva justificação.

5 Instruções de Deployment e Execução

O sistema foi preparado para ser executado localmente, sem necessidade de configurar servidores externos complexos.

5.1 Requisitos Mínimos

Para executar o projeto, são necessários os seguintes requisitos:

- Java Development Kit (JDK) 17 ou superior.
- Node.js 20.x ou superior.
- npm instalado.

5.2 Configuração do Servidor Backend

Para executar o backend, devem ser seguidos os seguintes passos:

1. Aceder à pasta do servidor:

```
cd servidor
```

2. Criar ou configurar o ficheiro `env.properties` na pasta `servidor/`. Este ficheiro é usado para guardar configurações sensíveis, como a chave da API Gemini.

```
gemini.api.key=SUA_CHAVE_GEMINI_AQUI
```

O projeto consegue iniciar sem esta chave, mas a geração automática de podcasts só fica disponível quando a chave da API Gemini é configurada.

3. Executar o servidor Spring Boot com o Maven Wrapper.

No Windows:

```
.\mvnw.cmd spring-boot:run
```

No Linux/macOS:

```
chmod +x mvnw  
./mvnw spring-boot:run
```

4. Depois de iniciado, o backend fica disponível em:

```
http://localhost:8080
```

A partir desse endereço, também ficam disponíveis os seguintes serviços:

- Consola da base de dados H2: `http://localhost:8080/h2-console`
JDBC URL: `jdbc:h2:./data/podcastia`
Utilizador: `sa`
Palavra-passe: vazia
- Swagger UI: `http://localhost:8080/swagger-ui/index.html`

5.3 Configuração da Interface Frontend

Para executar o frontend, devem ser seguidos os seguintes passos:

1. Aceder à pasta do frontend:

```
cd frontend
```

2. Instalar as dependências definidas no ficheiro `package.json`:

```
npm install
```

3. Iniciar o servidor de desenvolvimento Vite:

```
npm run dev
```

4. Depois de iniciado, o frontend fica disponível no browser em:

```
http://localhost:5173
```

6 Segurança

A segurança do Podcastia é implementada em múltiplas camadas, seguindo uma abordagem de defesa em profundidade, também conhecida como *defense in depth*. Esta abordagem permite combinar vários mecanismos de proteção, reduzindo o impacto caso uma das camadas falhe.

6.1 Modelo de Segurança

6.1.1 Autenticação JWT

O sistema utiliza tokens JWT com o algoritmo HMAC-SHA512, também identificado como HS512. Cada token tem validade de 24 horas e transporta os seguintes *claims*:

- **sub**: email do utilizador autenticado;
- **id**: identificador único do utilizador;
- **type**: papel do utilizador no sistema.

A transmissão do token é feita exclusivamente através do cabeçalho HTTP:

```
Authorization: Bearer <token>
```

Como os tokens não são armazenados em cookies, o sistema reduz o risco de ataques CSRF, uma vez que o browser não envia automaticamente o token em pedidos não autorizados.

6.1.2 Proteção de Passwords

As passwords dos utilizadores são armazenadas exclusivamente como hashes BCrypt, utilizando o fator de custo padrão do Spring Security.

Depois do registo, nenhum componente do sistema tem acesso à password em texto simples. Durante o processo de autenticação, a password introduzida pelo utilizador é comparada com o hash armazenado, sem ser necessário guardar ou recuperar a password original.

6.1.3 Controlo de Acesso

O controlo de acesso é implementado em dois níveis: declarativo e programático.

- **Controlo declarativo**: os endpoints de administração usam a anotação `@PreAuthorize("hasRole('USER_ADMIN')")` do Spring Security. Esta verificação é feita através de AOP antes da execução da lógica de negócio.
- **Controlo programático**: as operações sobre recursos verificam, no próprio código, se o utilizador autenticado tem permissão para executar a ação. Por exemplo, ao editar ou eliminar uma playlist, o sistema confirma se o utilizador autenticado é o proprietário desse recurso.

Esta combinação permite proteger tanto os endpoints sensíveis como as operações específicas sobre recursos pertencentes a cada utilizador.

6.2 Vulnerabilidades Conhecidas e Mitigações

Durante a análise do sistema foram identificadas algumas vulnerabilidades ou pontos de melhoria. A tabela seguinte apresenta a severidade, o estado e a mitigação prevista para cada caso.

Vulnerabilidade	Severidade	Estado e Mitigação
Chave JWT hardcoded em <code>JwtUtil</code>	Alta	Identificada. A chave deve ser movida para uma variável de ambiente antes do deploy em produção.
Logging de tokens JWT no frontend	Alta	Identificada. Os <code>console.log</code> que exponham tokens devem ser removidos em produção.
Mensagens de erro com detalhes internos	Média	Identificada. As mensagens de erro devem ser sanitizadas para evitar a exposição de detalhes internos da aplicação.
CORS configurado para <code>localhost</code>	Baixa	Resolvida para produção. A configuração deve ficar restrita a origens específicas no <code>SecurityConfig</code> .

Table 4: Vulnerabilidades de segurança identificadas.

7 Garantia de Qualidade e CI/CD

O projeto utiliza testes automatizados e pipelines de integração contínua para garantir a estabilidade do código e reduzir o risco de regressões.

7.1 Estratégia de Testes

A estratégia de testes está dividida entre backend e frontend, de acordo com as tecnologias usadas em cada parte do sistema.

7.1.1 Backend: JUnit 5 e Mockito

No backend, são usados testes unitários e testes de integração.

- **Testes unitários:** validam a lógica de negócio de forma isolada, recorrendo a mocks para simular repositórios e chamadas externas.
- **Testes de integração:** validam o funcionamento conjunto de vários componentes, incluindo a persistência na base de dados H2 de teste, as políticas de segurança e alguns endpoints da API.

Para executar os testes do backend localmente:

No Windows:

```
cd servidor
.\mvnw.cmd test
```

No Linux/macOS:

```
cd servidor
chmod +x mvnw
./mvnw test
```

7.1.2 Frontend: Vitest e Playwright

No frontend, são usados testes unitários, testes de componentes e testes end-to-end.

- **Testes unitários e de componentes:** são realizados com Vitest e verificam se os componentes React funcionam corretamente em diferentes estados.
- **Testes end-to-end:** são realizados com Playwright e simulam ações reais do utilizador no browser, como iniciar sessão, preencher o onboarding, usar o leitor de áudio e enviar mensagens.

Para executar os testes do frontend localmente:

```
cd frontend
```

Executar testes unitários em modo interativo:

```
npm run test
```

Executar testes unitários em modo CI:

```
npm run test:ci
```

Executar testes end-to-end com interface gráfica:

```
npm run test:e2e:ui
```

Executar testes end-to-end em modo CI:

```
npm run test:e2e
```

Verificar ou aplicar a formatação do código frontend:

```
npm run format
```

7.2 Análise Estática de Código

O projeto utiliza ferramentas de análise estática para detetar erros e manter um estilo de código consistente.

- **Checkstyle:** no backend, o Maven executa o Checkstyle com as regras definidas em `google_checks.xml`. Esta ferramenta verifica convenções de estilo, imports desnecessários e outros problemas no código Java. Se forem encontrados erros, a compilação pode falhar.
- **ESLint e Prettier:** no frontend, o ESLint é usado para detetar problemas no código JavaScript/React, enquanto o Prettier garante uma formatação uniforme. Estas ferramentas ajudam a manter o código mais limpo e consistente antes de cada commit.

7.3 Integração Contínua com GitHub Actions

O repositório inclui pipelines automáticas de integração contínua através do GitHub Actions. Estas pipelines ajudam a validar o código sempre que são feitas alterações.

- **Backend CI:** definida no ficheiro `.github/workflows/maven-ci.yml`. Esta pipeline corre em cada `push` e `pull request`. Utiliza uma máquina Ubuntu com Java configurado e executa o comando `./mvnw verify`, que compila o backend, corre os testes automatizados e executa a análise estática com Checkstyle.
- **Frontend CI:** definida no ficheiro `.github/workflows/frontend-ci.yml`. Esta pipeline corre quando existem alterações na pasta `frontend/`. Configura o Node.js 20, instala as dependências com `npm ci` e executa o linter, a formatação, os testes unitários e os testes end-to-end através dos comandos `npm run lint`, `npm run format`, `npm run test:ci` e `npm run test:e2e`.

8 Gestão do Projeto

8.1 Metodologia

O desenvolvimento do Podcastia seguiu uma metodologia ágil baseada em Scrum, com sprints semanais geridas através da plataforma Jira. O projeto foi organizado no Jira com a chave `POD`, estando disponível em:

<https://ualg-team-podcast.atlassian.net>

O backlog foi constituído por histórias de utilizador, também conhecidas como *User Stories*, e tarefas técnicas. Estas tarefas foram estimadas em *story points*, permitindo acompanhar o esforço previsto e o progresso da equipa ao longo do desenvolvimento.

No total, o projeto registou 111 pendências distribuídas por 12 sprints, tendo todas sido concluídas.

8.2 Progresso por Sprint

A tabela seguinte resume o progresso do projeto sprint a sprint, com base nos dados exportados do Jira em 24 de maio de 2026.

Sprint	Issues	Concluídos	Story Points	Foco Principal
Sprint 1	7	7	21,0	Configuração do projeto Spring Boot, modelos de dados e endpoints base.
Sprint 2	4	4	3,0	Backend de autenticação: criação de conta, login e gestão de contas.
Sprint 3	4	4	3,5	Frontend de autenticação, testes unitários e pipeline de CI/CD.
Sprint 4	4	4	5,5	Migração para JWT, logout e cabeçalho com nome de utilizador.
Sprint 5	11	11	11,2	Recomendações, cache, playlist, foto de perfil, OpenAPI e retoma de reprodução.
Sprint 6	n/a	n/a	n/a	Issues partilhadas com outros sprints, nomeadamente Sprint 5 e Sprint 7.
Sprint 7	9	9	13,0	Sistema de amigos no backend, testes e sidebar de detalhe do podcast.
Sprint 8	5	5	5,0	Endpoints de amizade: aceitar, recusar, listar e remover.
Sprint 9	3	3	3,5	Sistema de mensagens, reações e playlist diária automática.
Sprint 10	8	8	10,0	Dashboard de administração, gestão de conteúdo, frontend de mensagens e reações.
Sprint 11	8	8	3,5	Hot fixes, melhorias de UX, notificações e pesquisa sem login.
Sprint 12	9	9	9,0	Polimento visual, testes frontend, correção de bugs de áudio e pesquisa.
Total	111	111	88,2	Conclusão de todas as pendências registadas no Jira.

Table 5: Resumo do progresso por sprint, com base nos dados do Jira exportados em 24/05/2026.

8.3 Destaques por Sprint

8.3.1 Sprints 1 a 4: Fundações

Os primeiros quatro sprints estabeleceram as fundações do sistema. Nesta fase foi criada a estrutura do projeto Spring Boot, o modelo de dados e os endpoints iniciais.

Também foram desenvolvidas as funcionalidades base de autenticação, incluindo registo, login e alteração de password. Posteriormente, o sistema foi migrado para autenticação baseada em JWT.

Em paralelo, foi criada a interface de autenticação no frontend e configurada a pipeline de CI/CD com GitHub Actions.

8.3.2 Sprint 5: Motor Central

O quinto sprint foi o mais denso em termos técnicos, com 11 tarefas concluídas. Nesta fase foram implementados o algoritmo de recomendações por histórico, a cache de feed, o endpoint agregador da homepage, a retoma de reprodução e a criação de playlists.

8.3.3 Sprints 7 e 8: Sistema Social

Os sprints 7 e 8 focaram-se no sistema de amigos. Foram implementados os endpoints para enviar, aceitar, recusar, cancelar e remover ligações de amizade.

Também foram desenvolvidos os respetivos testes unitários e de integração. No frontend, foi criada a sidebar de detalhes do podcast e a navegação direta para o ecrã de player.

8.3.4 Sprints 9 e 10: Comunicação e Administração

O sprint 9 introduziu o sistema de mensagens privadas e de reações, bem como a automatização da playlist diária.

O sprint 10 completou o painel de administração, incluindo o dashboard de estatísticas e a gestão de conteúdo. Também foram desenvolvidas as funcionalidades de comunicação no frontend.

8.3.5 Sprints 11 e 12: Qualidade e Polimento

Os dois últimos sprints foram dedicados à correção de bugs, melhorias de experiência de utilizador e polimento visual.

Destacam-se a correção do estado do áudio ao navegar entre rotas, a implementação de testes de frontend com Playwright e a refatorização das páginas de playlists, tendências e homepage.